EuroMLSys ‘26, Edinburgh

Where the Time Goes

Analysis of a Public LLM Serving System

Büşra Karatay Demiray, Ehsan Yousefzadeh-Asl-Miandoab,
Pınar Tözün, Benoît Garbinato, Pamela Delgado

HEIG VD HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD
Hes·so
IT UNIVERSITY OF COPENHAGEN
Unil.
Swiss National Science Foundation

MOTIVATION

Why characterize a real LLM workload?

LLM Serving is critical day-to-day infrastructure
We need better *scheduling, autoscaling, resource allocations*

The disconnect

Current studies mostly rely on **synthetic benchmarks**

Production workloads are ***unknown***

Result: *misallocated optimization effort.*

Public production traces lack:

- phase-level timing breakdown
- per-model distributions
- cache-hit signals
- sufficient time coverage

Our contribution

1. **Five-month production trace**
   from a public, open-source serving system
2. **Workload characterization**
   identifying where latency originates in the pipeline
3. **Concrete recommendations**
   grounded in observed bottlenecks, not assumptions

THE DATASET

Overview

Pipeline from Public AI: A nonprofit, open-source service to make public and sovereign AI models more accessible

Users → **Open WebUI / API Calls** (chat / sessions) → **LiteLLM** (gateway · trace logged here) → **vLLM backends** (CSCS and other providers)

Apertus: a fully open LLM, developed by EPFL, ETH Zurich and CSCS as part of the Swiss AI Initiative

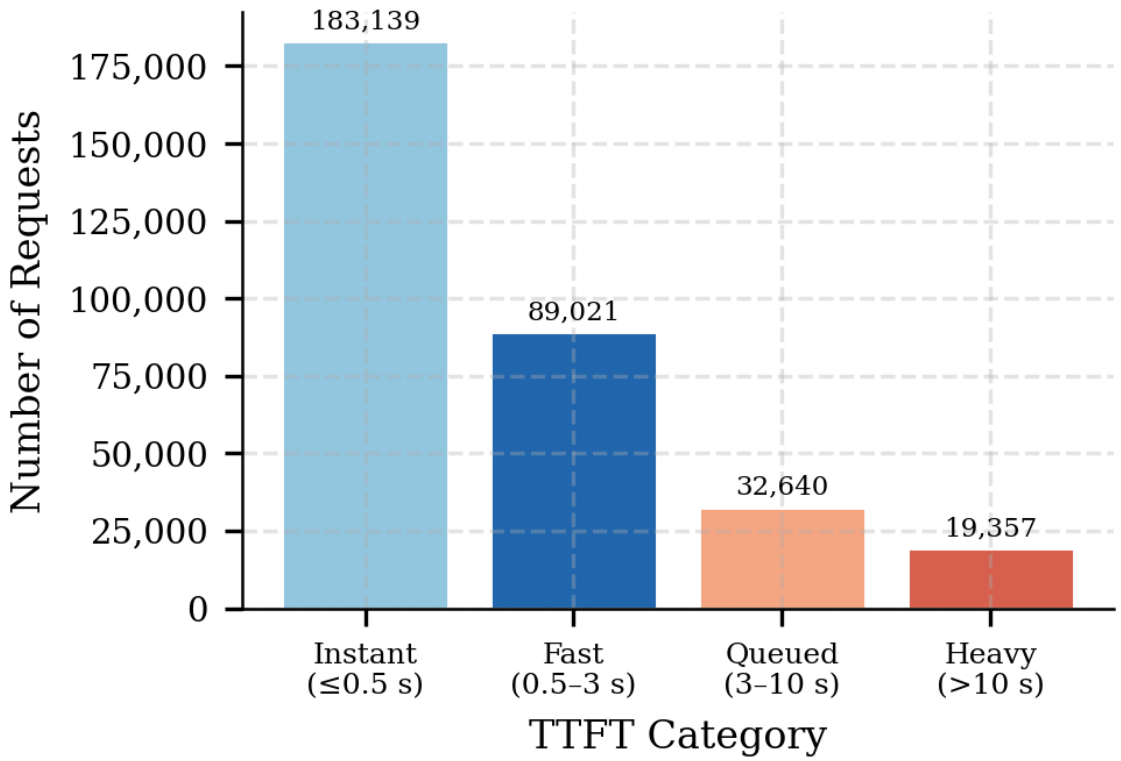
337K	5	Apertus	78 / 22
successful requests	months (Sep '25 – Jan '26)	2 variants (8B and 70B)	% traffic split (8B / 70B)

Per request: *timestamps (start/completion start/end) · tokens (prompt/completion/total) · cache-hit · turns*

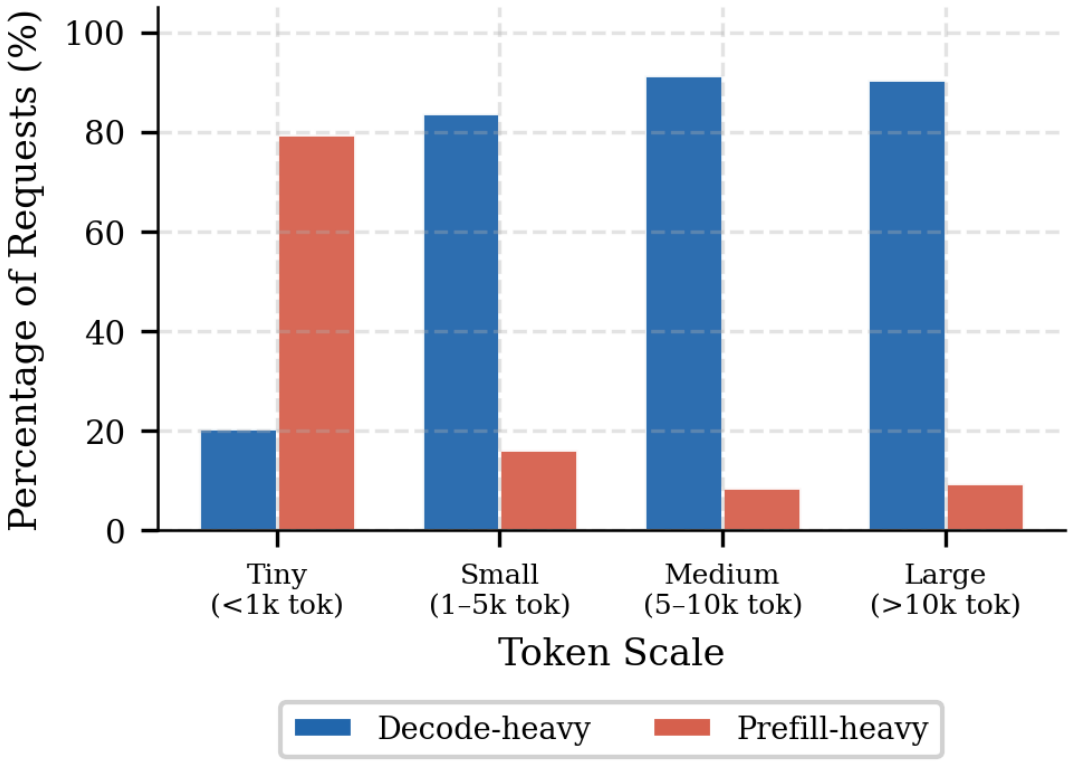
Trace release: anonymized trace + open Apertus weights + open-source framework constitutes an end-to-end pipeline.

FINDING 1

TTFT is dominated by queueing, not prefill compute



(a) Requests by TTFT category, 1 in 6 substantially queued

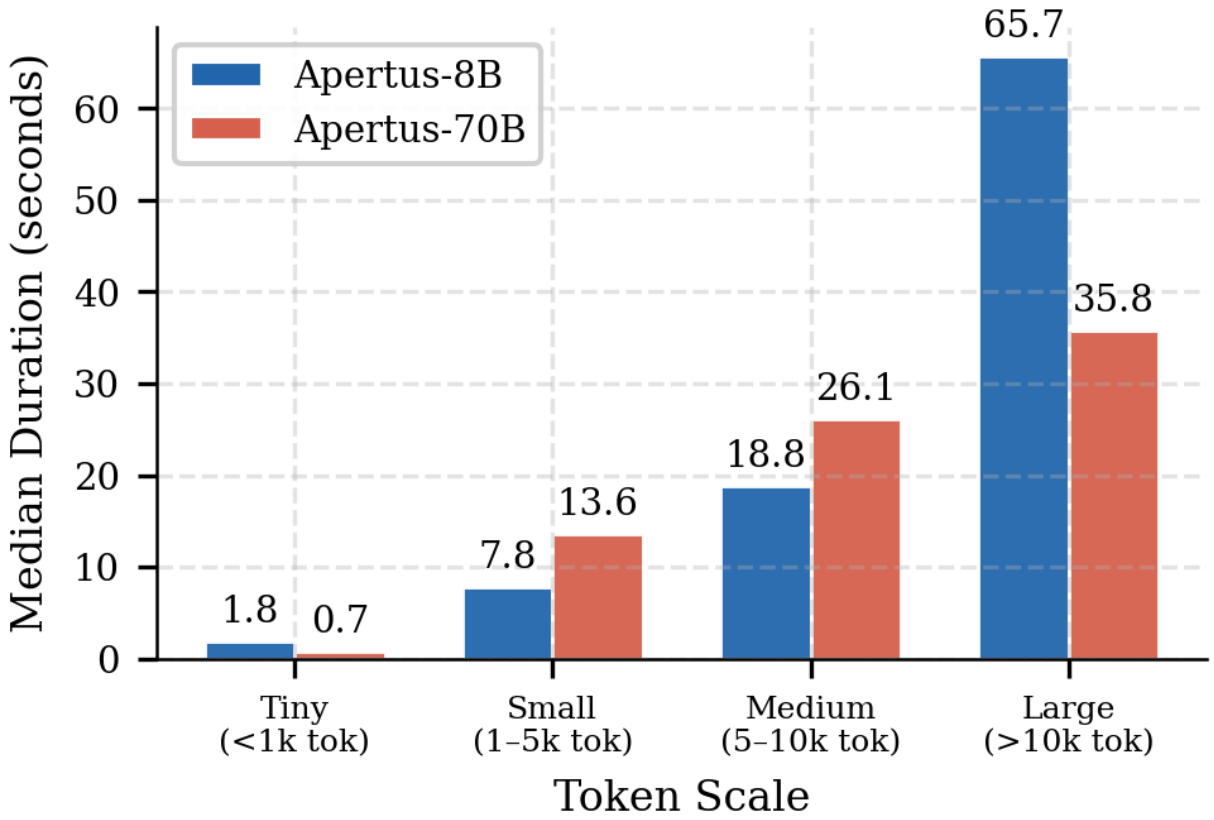


(b) Phase split by token scale, 80% of tiny requests are prefill-heavy

Tiny requests should compute in milliseconds, yet many start in seconds. **TTFT ≠ prefill.**

FINDING 2

Model size does not translate into user-perceived latency



Median end-to-end duration by model and token scale

What the user sees

For tiny and large requests, the 70B is actually **faster** than the 8B.

Compute is not irrelevant but dominated by system level factors;

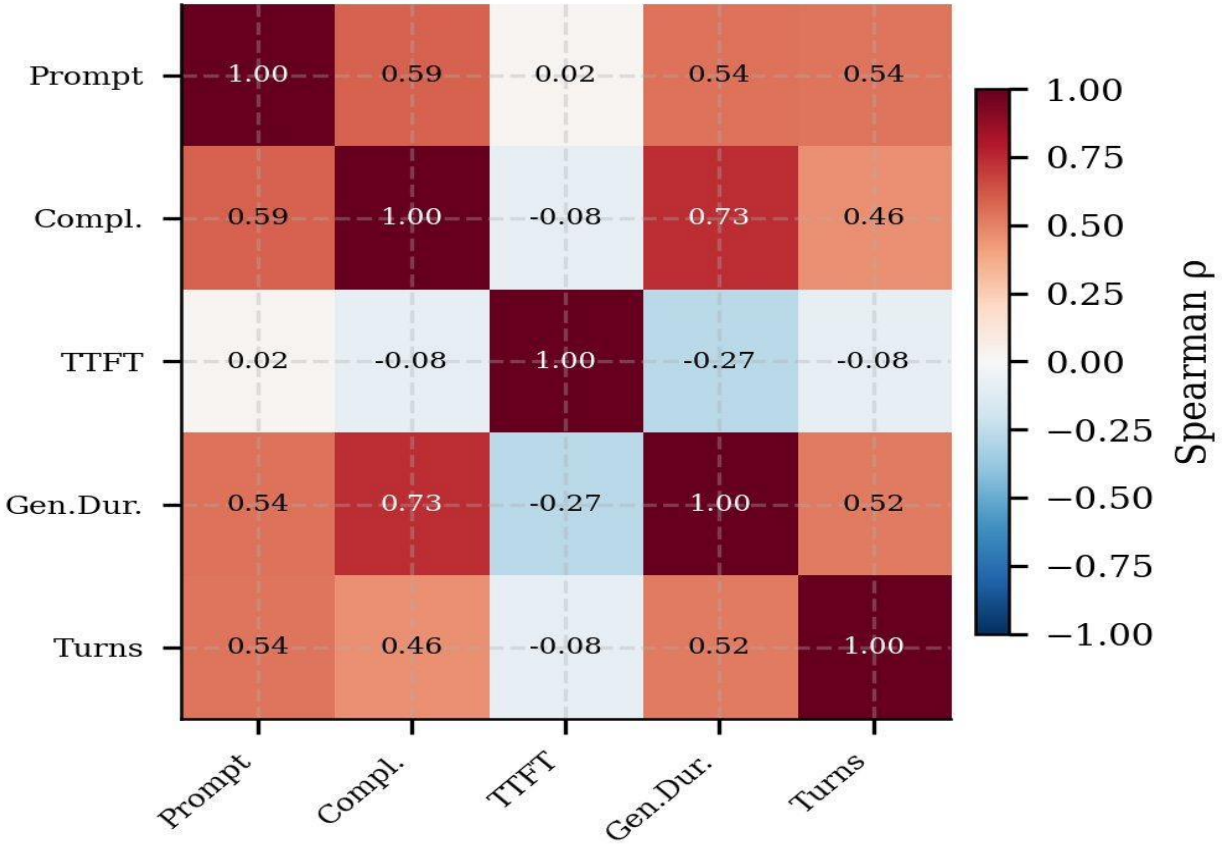
- Queuing
- Resource allocation
- Traffic

The compute advantage is consumed by queueing. (%78 to 8B, %22 to 70B)

In this stack, system-level effects mediate model size, routing on size as a latency proxy is unreliable.

FINDING 3

Arrival time features are weak predictors



The heatmap shows the correlation between the features

- *Each cell shows the correlation between two features.*
- *Darker red / blue = stronger positive/negative relationship.*
- *Near-white ≈ no relationship.*

The signals a scheduler can actually use

0.02	**Prompt Tokens ↔ TTFT** *essentially zero*
0.54	**Prompt Tokens ↔ Generation Duration** *moderate, noisy*
0.52	**Conversation Turns ↔ Generation Duration** *moderate, noisy*
0.73	**Completion Tokens ↔ Generation Duration** *but unobservable at arrival*

Point-estimate schedulers operate on weak signal, predictive policies need misprediction-aware fallbacks.

IMPLICATIONS

Treat queueing as a first-class bottleneck

1

Size-aware priority

Prioritize tiny / small requests, most of their latency is removable queueing.

2

Model-agnostic decisions

Route on observed bottleneck regime, not on parameter count as a latency proxy.

3

Dynamic capacity

Allocate along the daily traffic profile to flatten queue formation at peak hours.

4

Adaptive batching

React to live queue depth and request mix, avoid throughput-only policies that punish small requests.

Common theme: *for a large fraction of requests, time waiting > time computing.*

THANK YOU · TAKEAWAYS

Thank you for listening!

01

TTFT is queueing.

Not prefill computing. Tiny requests wait seconds for milliseconds of work.

02

Model size ≠ user latency.

Smaller, simpler model is slower in this setup; system-level effects dominate the parameter gap.

03

Predict with caution.

Arrival features are weakly or moderately correlated with duration. Schedulers need misprediction-aware fallbacks.

Trace: https://github.com/heigvd-software-engineering/apertus-serving-analysis

For questions: busra.karatayd@hes-so.ch